

MathLab 3: Module 3

Course Reference: *Data Structures and Algorithms in Python*
Goodrich, Tamassia, Goldwasser (Wiley)

Module 3: Graphs and Traversal Algorithms

Weeks 5–6 Lab: 3 hrs/week Quiz: end of Week 6

Corresponding Chapter(s) in the Book

Chapter 14 – *Graph Algorithms*: §14.1 (Graphs, the Graph ADT), §14.2 (Data Structures for Graphs – edge list, adjacency list, adjacency map, adjacency matrix), §14.3 (Graph Traversals – DFS §14.3.1–14.3.2, BFS §14.3.3), §14.6.1 (Weighted Graphs, introductory).

Key Concepts – Brief Revision

- A graph $G = (V, E)$ consists of vertices and edges; edges may be directed/undirected and weighted/unweighted.
- Common representations: edge list ($O(m)$ space, slow adjacency queries), adjacency list/map ($O(n + m)$ space, efficient neighbor iteration), adjacency matrix ($O(n^2)$ space, $O(1)$ edge lookup).
- DFS explores as deep as possible before backtracking (stack-based, often via recursion); it is used to detect connectivity, cycles, and to compute a DFS forest.
- BFS explores level by level (queue-based); it computes shortest paths in *unweighted* graphs and can detect connected components and bipartiteness.
- A graph's connected components can be found via repeated DFS/BFS from unvisited vertices.
- A graph is bipartite if its vertices can be 2-colored such that no edge connects two vertices of the same color; this can be tested via BFS/DFS with alternating colors.
- SageMath's `Graph()` and `DiGraph()` classes provide built-in construction, visualization (`.plot()`, `.show()`), and many graph-theoretic algorithms useful for verifying hand-written implementations.

Home Assignment – 30 Questions

1. Implement a graph using an adjacency list represented as a Python dictionary of lists, supporting `add_vertex`, `add_edge`, and `neighbors(v)`.

2. Implement the same graph ADT using an adjacency matrix (2D list), and write a converter function `adj_list_to_matrix` and its inverse.
3. For a graph with n vertices and m edges, empirically compare the memory footprint (using `sys.getsizeof` recursively, or a rough estimate) of your adjacency list vs. adjacency matrix implementations for a sparse graph ($m \approx n$) and a dense graph ($m \approx n^2/2$).
4. Build a SageMath `Graph` object for the Petersen graph (`graphs.PetersenGraph()`), visualize it with `.show()`, and print its vertex and edge counts, comparing against your own adjacency-list construction of the same graph.
5. Implement DFS from scratch (recursive version) on your own adjacency-list graph class, returning the DFS tree as a dictionary of `child: parent` pairs.
6. Implement DFS iteratively using an explicit stack (not recursion), and verify on 5 random graphs that it visits the same set of vertices as your recursive version (though not necessarily in the same order).
7. Implement BFS from scratch, returning both the visiting order and a dictionary of shortest hop-distances from the source vertex.
8. Using your BFS implementation, write a function `shortest_unweighted_path(G, s, t)` that reconstructs the actual path (sequence of vertices) from s to t , not just its length.
9. Write a function `connected_components(G)` that returns a list of vertex sets, one per connected component, using repeated BFS or DFS. Test it on a graph you construct with at least 3 disconnected pieces.
10. Using SageMath, generate a random graph `graphs.RandomGNP(30, 0.05)` and use your `connected_components` function to find its components; cross-check the count against `G.connected_components_number()`.
11. Implement a function `is_bipartite(G)` that attempts a 2-coloring via BFS and returns `(True, coloring)` or `(False, None)` if an odd cycle is found.
12. Test your `is_bipartite` function on (a) a cycle graph C_6 , (b) a cycle graph C_5 , and (c) the complete bipartite graph $K_{3,3}$, explaining the expected result for each using SageMath's `graphs.CycleGraph` and `graphs.CompleteBipartiteGraph`.
13. Write a function that detects whether an *undirected* graph contains a cycle, using DFS and parent-tracking to distinguish a back edge from revisiting the immediate parent.
14. Write a function that detects whether a *directed* graph contains a cycle, using DFS with a “currently in recursion stack” marker (distinguishing tree, back, forward, and cross edges conceptually).
15. Implement a function to compute all vertices reachable from a given source in a directed graph, and compare its result to SageMath's `DiGraph.breadth_first_search(source)`.
16. Given an adjacency list, write a function that computes the degree of every vertex, and verify the handshake lemma ($\sum_v \deg(v) = 2m$ for undirected graphs) on 5 randomly generated graphs.
17. Write a function `is_connected(G)` using a single BFS/DFS pass, and test its correctness on both connected and disconnected SageMath-generated graphs.

18. Implement Euler-tour style DFS (as discussed in the book’s tree traversal analogy, §8.4.6) adapted to general graphs, printing a “preorder” and “postorder” visitation timestamp for each vertex.
19. Using timestamps from Q18, classify every edge of a directed graph as a tree, back, forward, or cross edge, and verify against SageMath’s DFS edge classification if available, or manually on a small hand-built example.
20. Write a function that finds the shortest cycle length in an unweighted undirected graph containing at least one cycle (hint: BFS from each vertex).
21. Model a small social network (at least 8 people, with friendships as edges) as a graph, and write functions to find (a) the person with the most friends, (b) whether two given people are connected through mutual friends within 2 hops.
22. Implement a maze solver: represent a 2D grid maze (with walls) as a graph where free cells are vertices and adjacent free cells are edges, then use BFS to find the shortest path from entrance to exit.
23. Given a weighted graph (as an adjacency list of (`neighbor`, `weight`) pairs), write a function that computes the total weight of a given path (list of vertices), raising an error if consecutive vertices are not adjacent.
24. Using SageMath, construct a random weighted graph and use `.plot(edge_labels=True)` to visualize it with weights shown, then manually verify by inspection that your adjacency-list weighted representation matches the drawing.
25. Write a function that, given a graph, determines whether it is a tree (connected and $|E| = |V| - 1$), and test it on both tree and non-tree SageMath graphs.
26. Implement topological-sort-readiness check: write a function that determines whether a directed graph is a DAG (no cycles) using your Q14 cycle detector, without yet performing the actual sort.
27. Write a program that reads a list of edges from a text file (you create the file) and builds a graph, reporting the number of vertices, edges, and connected components – a simple “graph summary” tool.
28. Implement a function to compute the diameter of a small connected unweighted graph (the maximum shortest-path distance between any pair of vertices), using repeated BFS from every vertex, and verify against SageMath’s `G.diameter()`.
29. Given two graphs of the same size (as adjacency lists), write a function that checks whether they *might* be isomorphic by comparing sorted degree sequences (a necessary but not sufficient condition), and test it against SageMath’s `G1.is_isomorphic(G2)` on 3 example pairs.
30. Design and implement a simple “six degrees of separation” explorer: given a graph and two vertices, use BFS to compute their distance, and report the shortest connecting path for 5 different vertex pairs in a SageMath-generated random graph.